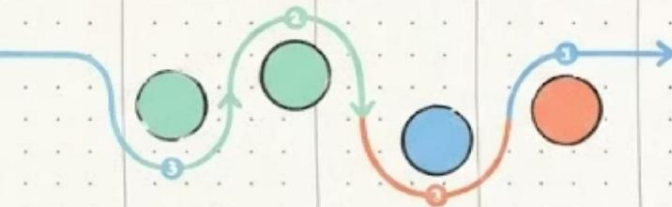


Gestão do Ciclo de Vida da Aplicação

Processos de Desenvolvimento de Software (XP)

Professor Romário da Silva Borges



Problemas frequentes em projetos de software



Mudanças constantes

Requisitos mudam no meio do projeto e tudo precisa ser refeito



Atrasos e estouros

Prazos não são cumpridos e o orçamento ultrapassa o previsto



Falhas de comunicação

Equipe, cliente e stakeholders não falam a mesma língua



Defeitos tardios

Problemas críticos são descobertos apenas na entrega final



Retrabalho excessivo

Funcionalidades precisam ser reescritas porque o entendimento estava errado



Produto não usado

O sistema é entregue, mas não atende às reais necessidades do usuário

Extreme Go Horse

✗ eXtreme Go Horse (XGH)

- "Pensou, não é XGH"
- "Se está funcionando, não mexa"
- "Funcionou no meu computador, então está pronto"
- Sem testes, sem documentação, sem planejamento
- Corrigir problemas só quando aparecem



Extreme Go Horse

XeXtreme Go Horse (XGH)

Alguns “mandamentos” do XGH são:

1. Quanto mais XGH você faz, mais precisará fazer.
2. Os erros só existem quando aparecem.
3. Esteja preparado para pular fora quando o barco começar a afundar, ou coloque a culpa em alguém ou algo.
4. Se tiver funcionando, não rela a mão.



Método correto vs. método Go Horse



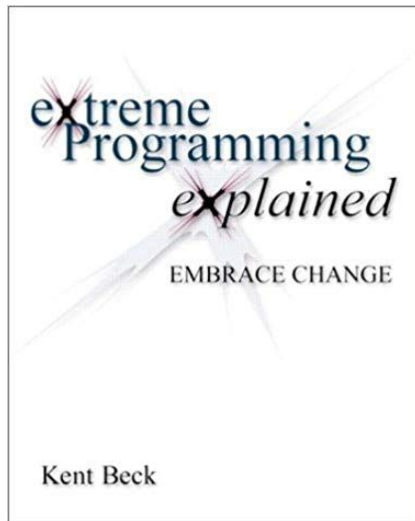
Extreme Programming (XP)

The background of the slide features a light blue and white gradient. Overlaid on this are several wavy, horizontal lines in shades of purple and blue. These lines appear to be composed of or contain snippets of computer code, likely in a scripting language like JavaScript or PHP, which is blurred and integrated into the wave patterns.

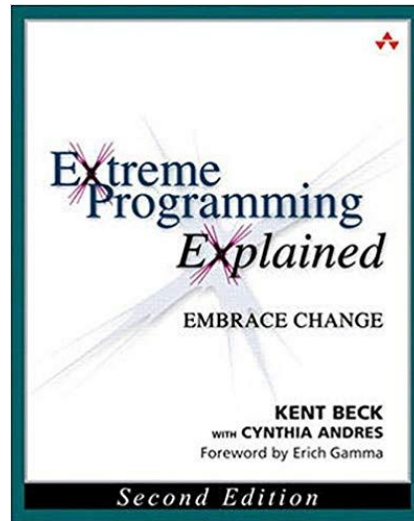
Extreme Programming



Kent Beck



1999



2004

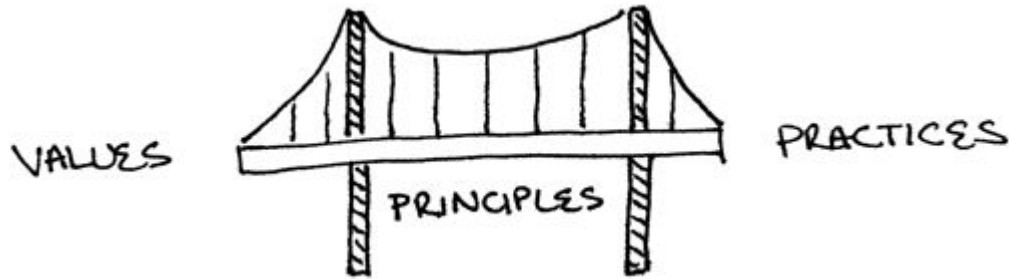
XP = Valores + Princípios + Práticas

Valores

- Comunicação
- Simplicidade
- Feedback
- Coragem
- Respeito
- Qualidade do Ambiente de Trabalho

Valores ou cultura são fundamentais em software!

XP = Valores + Princípios + Práticas



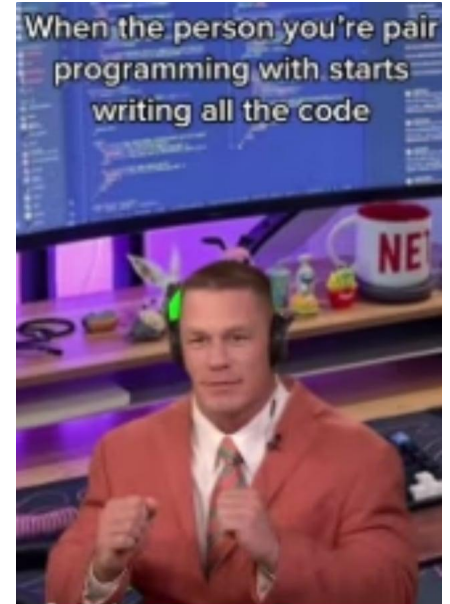
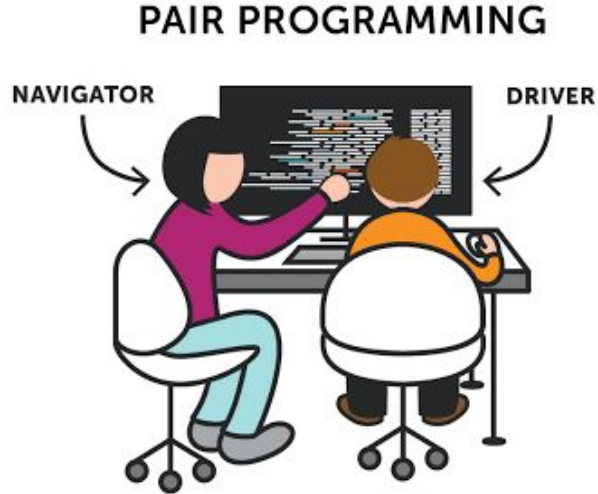
Princípios

- Humanidade (*Peopleware*)
- Viabilidade econômica
- Melhorias Contínuas
- Falhas Acontecem
- Baby Steps
- Responsabilidade

XP = Valores + Princípios + Práticas

Práticas sobre o Processo de Desenvolvimento	Práticas de Programação	Práticas de Gerenciamento de Projetos
Representante dos Clientes Histórias de Usuário Iterações Releases Planejamento de Releases Planejamento de Iterações Planning Poker Slack	Design Incremental Programação Pareada Testes Automatizados Desenvolvimento Dirigido por Testes (TDD) Build Automatizado Integração Contínua	Ambiente de Trabalho Contratos com Escopo Aberto Métricas

Pair Programming

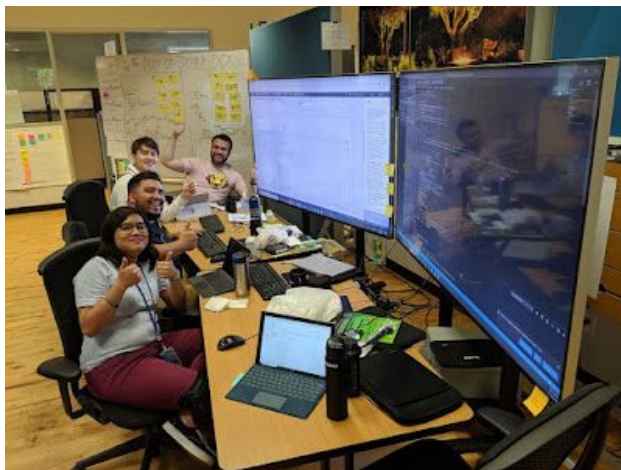


Exercício

- Quais as vantagens de pair programming?
- E a principal desvantagem?

Mob Programming

- Time inteiro trabalhando na mesma tarefa de programação
- Qual o principal “caso de uso” de mob programming?



Pull Requests + Revisão de Código (assíncrona)

estacionamento.java

Outdated

2

+

3

+

public class Estacionamento {

4

+

5

+

public Hashtable<String, String> veiculos;

 **mtov** 5 days ago

😊

...

Sugestão: "ocultar" essa tabela hash dos clientes da classe, tornando-a privada. E, também, criar um método "estaciona" que deve ser chamado toda vez que um novo cliente quiser estacionar um veículo.

 **alinedtorres** 3 days ago

Author

Owner

😊

...

Agradeço a sugestão! Acabei de realizar as alterações sugeridas.

 **mtov** now

😊

...

Obrigado, LGTM!

 Reply...

Programação Pareada e Revisão de Código com IA

Introducing GitHub Copilot: your AI pair programmer

Today, we're launching a technical preview of GitHub Copilot, a new AI pair programmer that helps you write better code.

Nat Friedman · @nat

June 29, 2021 | Updated February 23, 2022 | ⌚ 1 minutes

Share:



Junho de 2021, logo antes do ChatGPT

Pareamento com IA (via auto-complete)

JS test.js 1 ●

JS test.js >  calculateDaysBetweenDates

```
1  function calculateDaysBetweenDates(begin, end) {  
    var beginDate = new Date(begin);  
    var endDate = new Date(end);  
    var days = Math.round((endDate - beginDate) / (1000 * 60 * 60 * 24));  
    return days;  
  }  
2
```

<https://code.visualstudio.com/docs/copilot/ai-powered-suggestions>

Pareamento com IA (via revisão de código)

jobs:

....

- name: Automatic PR Review

- uses: anthropics/claude-code-action@beta

- with:

- anthropic_api_key: \${{ secrets.ANTHROPIC_API_KEY }}

- timeout_minutes: "60"

- direct_prompt: |

- Please review this pull request and provide comprehensive feedback.

- Focus on:

- Code quality and best practices
 - Potential bugs or issues
 - Performance considerations
 - Security implications
 - Test coverage
 - Documentation updates if needed

Executado automaticamente sempre que um novo PR for aberto em um repositório



Bringing Code Review to Claude Code

Claude Code now has a thorough, agent team-based review system, modeled on the one we run at Anthropic. Available in research preview.

<https://claude.com/blog/code-review>

	Síncrono	Assíncrono
Humanos	Programação Pareada	Revisão de Código
Automatizada	GitHub Copilot	Revisão de Código com IA

Contratos de Software

- Contratos de software podem ser de dois tipos:
 - Escopo Fechado
 - Escopo Aberto (defendidos por XP)

Contratos com Escopo Fechado

- Cliente: requisitos ("fecha escopo")
- Empresa: preço + prazo

Contratos com Escopo Aberto

- A empresa CONTRATADA fornecerá [x] programadores para desenvolver um sistema [y] para a empresa CONTRATANTE, ao custo de [z] reais por mês.
- Este contrato será renovado a cada mês, a não ser que uma das partes decida cancelá-lo.

Contratos com Escopo Aberto

- Exige maturidade e acompanhamento do cliente
- Vantagens:
 - Privilegia qualidade
 - Não vai ser enganado ("entregar por entregar")
 - Pode mudar de fornecedor

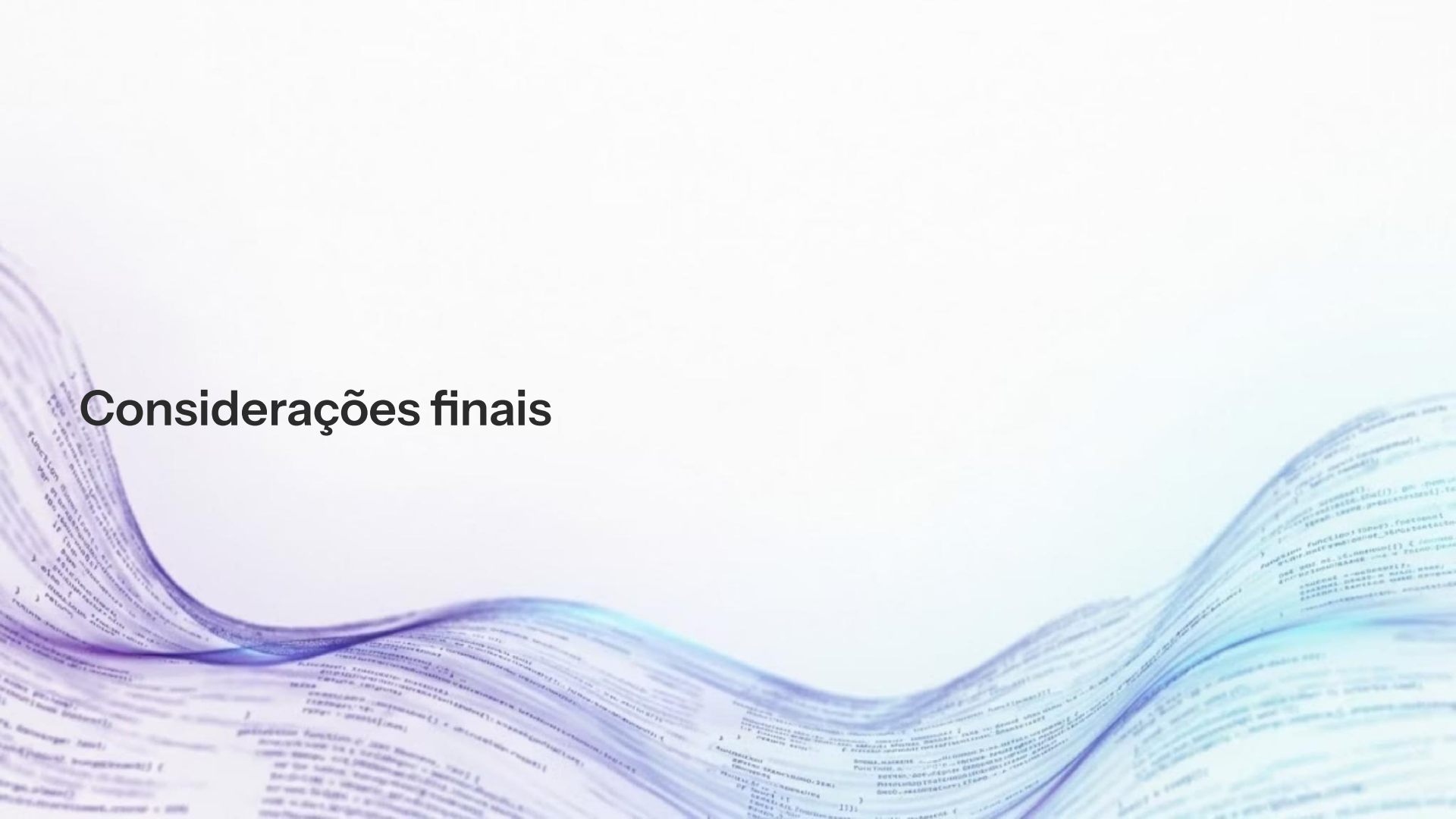
1. Por que XP é um método voltado apenas para projetos de software?

Atividades

The background features a series of fluid, overlapping waves in shades of light blue and lavender. These waves create a sense of motion and depth. Scattered throughout the background are faint, semi-transparent snippets of code, likely in JavaScript or a similar language, which are partially obscured by the waves and the main title.

1. Qual a duração ideal de uma iteração?
2. Como escolher o representante dos clientes?
3. Como definir a velocidade do time?
4. A história X depende da história Y, mas o representante dos clientes priorizou X antes de Y. O que devo fazer?

Considerações finais

The background features a series of fluid, overlapping waves in shades of light blue and lavender. These waves create a sense of movement and depth. Scattered throughout the background are faint, semi-transparent snippets of code in various programming languages, including JavaScript, Python, and C++, which blend into the overall aesthetic.

Vantagens, Limitações e Desafios do XP

Potenciais vantagens

- Testes e refatoração contínuos melhoram estabilidade e manutenção
- Design simples reduz complexidade desnecessária
- Pequenas entregas geram valor mais cedo
- Colaboração aumenta visibilidade e compartilhamento de conhecimento
- Participação frequente do cliente reduz risco de construir a solução errada

Os benefícios dependem da qualidade da aplicação das práticas e do contexto organizacional.

Limitações e desafios

- Participação frequente do cliente pode ser difícil em algumas organizações
- Programação em pares exige adaptação cultural
- Testes automatizados demandam investimento inicial
- Integração contínua depende de infraestrutura e disciplina
- Documentação reduzida não significa ausência de documentação
- Mais difícil em sistemas regulados, equipes muito grandes ou com cliente indisponível

Professor Marco Tulio Valente

Slides disponibilizados pelo Prof. Marco Tulio Valente. Esta apresentação contém adaptações da apresentação original que está disponível na seguinte página: <https://engsoftmoderna.info>.

BEZERRA, E.

Princípios de Análise e Projeto de Sistemas com UML, 2a ed., Editora Campus/Elsevier: Rio de Janeiro, 2007

PAULA, Filho W. P.

Engenharia de Software: Fundamentos, Métodos e Padrões, 3a ed. Ed. LTC: Rio de Janeiro, 2009.

HORSTMANN, C.

Padrões e Projeto Orientados a Objetos, 2 Edição, Editora Bookman: Porto Alegre, 2007

SOMMERVILLE, I.

Engenharia de Software. 8a ed., Ed. Pearson: São Paulo, 2008.

PRESSMAN, R.

Software Engineering: A Practitioner's Approach. 5a ed. McGraw Hill : São Paulo, 2000

